
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative framework for probabilistic forecasting in multivariate timeseries settings in which historical measurements, together with optional future covariates, induce a learned conditional law over prospective trajectories. Rather than modeling the full predictive horizon as a monolithic object, we decompose the joint distribution of future observations into an ordered collection of conditional densities, each represented by a shared flow that transports a simple base measure to the next-observation distribution under the relevant covariate context; this construction is trained through the flow matching (FM) paradigm, thereby permitting scalable, simulation-free estimation of the requisite transformations. By coupling this autoregressive factorization with the FM objective, FLOWTIME preserves the characteristic advantages of autoregressive forecasting—notably strong extrapolative behavior, compact parameterization, and calibrated uncertainty quantification—while inheriting from modern transport-based generative modeling the capacity to represent intricate, multi-modal conditional distributions. Empirical evaluations on several stochastic dynamical systems and real-world forecasting benchmarks demonstrate the effectiveness of the proposed model.

1 INTRODUCTION

Probabilistic timeseries forecasting constitutes a central problem in contemporary machine learning: from finite and often noisy observational histories, the forecaster must infer not merely a point estimate of future system evolution, but an entire predictive distribution over possible trajectories. The resulting task is consequential across a broad spectrum of scientific, industrial, and societal domains in which decisions depend on calibrated anticipations of dynamical behavior (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

A growing body of work therefore recasts forecasting as conditional generative modeling, using deep generative architectures to sample future trajectories from probability laws conditioned on past observations (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Within this formulation, diffusion and score-based models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have recently advanced performance on several forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kolloviah et al., 2023; Meijer & Chen, 2024); nevertheless, their reliance on iterative denoising procedures generally imposes substantial computational burdens during both model estimation and downstream sampling.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) has emerged as a generative modeling paradigm that generalizes and subsumes diffusion-based constructions while expanding the available design space for probability transport. In contrast to diffusion models, which approximate generation through stochastic denoising along a discretized trajectory, FM learns deterministic probability paths that transform arbitrary base distributions into target distributions through continuous normalizing flows. With suitably designed paths, FM obviates handcrafted noise schedules and lengthy ancestral sampling chains, thereby enabling more efficient training and inference without abandoning expressive distributional modeling.

Recent applications of FM to timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kolloviah et al., 2024) have reported strong empirical performance and improved computational efficiency relative to diffusion-based alternatives. Yet existing approaches typically

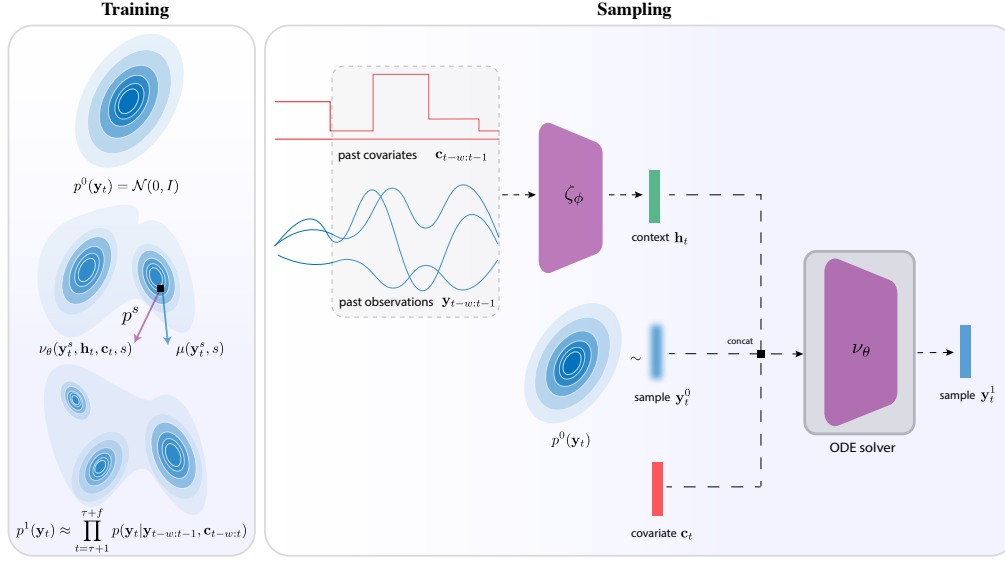


Figure 1: Schematic overview of the proposed training and sampling procedure. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ is constructed between a base distribution p^0 and the target distribution p^1 , with the path generated by a velocity field μ . The model estimates μ by regressing a neural network ν_θ on samples y_t^s from the path at flow step s , conditioned on a context vector h_t encoding past observations and covariates together with the current covariate c_t . At inference time, target-distribution samples are obtained by first sampling from p^0 and then integrating the learned velocity field with an ODE solver until $s = 1$.

learn, in a single step, the conditional distribution of an entire fixed-length future window given a fixed-length context window. Although this non-autoregressive parameterization can accelerate training and sampling, it may also yield poor extrapolation outside the training distribution and produce uncertainty estimates whose calibration deteriorates under distributional shift. Moreover, the model must optimize complex time-dependent probability paths along the flow dimension; this difficulty is explicitly recognized in (Kolloviev et al., 2024), where Gaussian-process-informed priors are introduced to simplify and expedite optimization, albeit at the cost of restricting the method to univariate forecasting.

We propose FLOWTIME, a deliberately simple FM-based alternative that scales probabilistic forecasting to high-dimensional multivariate datasets without recourse to such informed priors. Rather than transporting a complete future window simultaneously, FLOWTIME factorizes the forecast into a sequence of conditional distributions, so that each future time point is modeled given the relevant history and covariates through a shared flow from a simple base law to the next-observation distribution. This autoregressive organization both simplifies the associated optimization problem and endows the model with practical forecasting properties: variable-horizon generation, extrapolation beyond the training regime, and well-calibrated uncertainty propagation through time.

Across classical stochastic dynamical systems and real-world forecasting datasets, we show that FLOWTIME delivers strong empirical performance, thereby furnishing a practical and effective route to probabilistic timeseries forecasting with flow matching.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative modeling paradigm that enables continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) to be trained at unprecedented scale. CNFs represent data transformations as solutions to ordinary differential equations (ODEs), thereby inducing invertible mappings with tractable likelihoods.

As in CNFs, FM seeks a time-dependent diffeomorphic map on the data space Ω , referred to as a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transports a source sample $\mathcal{X}^0 \sim p$ into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ satisfying $\mathcal{X}^1 \sim q$ for a target distribution q , and it is obtained by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the velocity field of the flow and induces a probability path $(p^s)_{0 \leq s \leq 1}$, with each p^s a distribution over Ω and $p^0 = p$. The learning problem is therefore to identify a valid vector field μ whose terminal distribution satisfies $p^1 = q$.

Unlike conventional CNF training, in which the vector field is learned by likelihood maximization and therefore requires solving and differentiating through Eq. (1) during optimization, FM estimates a parameterized vector field ν_θ by regression against vector fields associated with fixed conditional probability paths. This construction yields the tractable conditional flow matching (CFM) objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After this objective is minimized, sampling from q proceeds by solving Eq. (1) with the learned ν_θ . Because training avoids gradient backpropagation through the ODE solver, FM supports stable and scalable optimization for high-dimensional generative tasks, including image and video modeling (Esser et al., 2024; Polyak et al., 2024). A variety of probability-path designs and conditioning strategies have been proposed, thereby unifying multiple transport-based generative models within a common framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given access to l historical measurements, including the current observation, denoted $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we seek probabilistic forecasts for the subsequent f values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, potentially conditional on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, given a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ containing m instances, our goal is to learn a sampler from the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

We factorize this conditional distribution autoregressively across future time steps under a Markov assumption of order w , yielding the decomposition

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ specifies the effective history length. Consequently, learning the forecasting law reduces to estimating the conditional distributions appearing in Eq. (3). Rather than adopting the maximum-likelihood objectives commonly used for autoregressive models, we estimate these conditionals through the flow matching framework (Lipman et al., 2022).

This autoregressive factorization confers several methodological advantages. First, it replaces the direct estimation of a high-dimensional joint forecast law with a family of comparatively simpler conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. Second, because each conditional can be trained under teacher forcing, the time-indexed training problems remain parallelizable despite the sequential sampling semantics. Third, the Markov window w provides an explicit mechanism for balancing representational capacity against computational cost. Finally, the resulting chain of conditionals affords an interpretable account of uncertainty propagation, since predictive uncertainty is injected and transformed step by step through the forecast horizon.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau + 1, \tau + f]$, we introduce a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ whose endpoints are $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, with $p_{\mathcal{D}}$ denoting the empirical distribution. The path is generated along the flow coordinate s by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ satisfying the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field selected to generate a flow whose induced densities satisfy the continuity equation.

To obtain a tractable learning objective, the marginal probability path can be expressed as a mixture over conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning variable drawn from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 of (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with the conditional paths $p^s(\mathbf{y}_t|\mathbf{z})$ are equivalent, for learning purposes, to the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we instantiate the coupling with $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ and $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are respectively sampled from the base and target distributions. The conditional probability path is then specified as

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with endpoint marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ as $\sigma^2 \rightarrow 0$. The associated conditional velocity field therefore reduces to the simple displacement $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction yields straight Euclidean probability paths with closed-form targets, which supports efficient training and sampling; once the velocity field has been learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by solving the ODE in Eq. (4) up to $s = 1$.

3.2 TRAINING

Analogously to the standard FM setting, we estimate this velocity field by regressing a neural network ν_θ , parameterized by θ , onto the conditional flow matching target in Eq. 2. To make this estimation compatible with the autoregressive forecasting structure, we define a context embedding $\mathbf{h}_t \in \mathbb{R}^h$ that summarizes the context window at time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural encoder with parameters ϕ . The resulting learning objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which is optimized by sampling from $\mathcal{D}$ an observation at time t together with its context window and covariates, and then jointly updating ϕ and θ through stochastic gradient descent on $\mathcal{L}(\theta, \phi)$. This procedure is analogous to teacher forcing (Williams & Zipser, 1989): during training, ground-truth historical observations, rather than the model’s own previous predictions, condition each forecast step, enabling parallel optimization over time indices while preventing training-time error accumulation. Algorithm 1 summarizes the autoregressive flow matching training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_t^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
 - 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
 - 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
 - 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
 - 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
 - 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
 - 7: Update θ, ϕ
 - 8: **end while**
 - 9: **return** ν_θ, ζ_ϕ
-

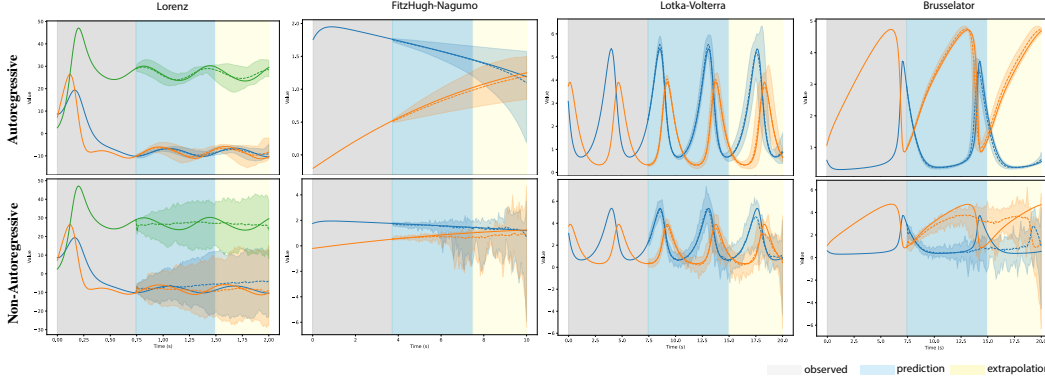


Figure 2: Representative forecast samples for autoregressive flow matching and standard flow matching across four stochastic dynamical systems. Solid curves denote ground truth, whereas dashed curves report the predictive mean together with the 95% confidence interval; both prediction and extrapolation regimes are visualized.

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
 - 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
 - 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
 - 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
 - 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$, $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$, $s \in [0, 1]$
 - 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
 - 7: **end for**
 - 8: **return** $\mathbf{Y}_f$
-

3.3 INFERENCE

After training, the model is used to produce probabilistic predictions for new observations $\mathbf{Y}_l$ and corresponding covariates $\mathbf{C}$ over the next f time points. We follow the rolling-window sampling procedure in Algorithm 2, generating each forecast autoregressively and then feeding it into the subsequent context; repeated execution of this procedure yields empirical predictive samples from which uncertainty quantiles can be estimated.

4 EXPERIMENTS

This section reports the empirical evaluation of FLOWTIME, comparing the proposed model with relevant baselines on both simulated stochastic dynamical systems and real-world forecasting datasets.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on forecasts from five stochastic dynamical systems: stochastic variants of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically integrating the corresponding stochastic differential equations under randomly sampled initial conditions and Brownian motion realizations; additional details on the dynamical specifications and data-generation protocol appear in Appendix A.1.

Implementation details In the dynamical-systems experiments, ζ_ϕ was instantiated as a two-layer bi-directional LSTM with 64 hidden units, whereas ν_θ was implemented as an MLP with three hidden layers of width 64. The flow step s was encoded by Fourier positional embeddings into a

16-dimensional representation using a fixed frequency set, and the context length w was set equal to the prediction length. Models were trained by stochastic gradient descent with the Adam optimizer, a learning rate of 0.003, and a batch size of 128; convergence required approximately ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the contribution of the autoregressive factorization in FLOWTIME, we compared against a non-autoregressive flow-matching baseline that directly represents the joint conditional distribution of future observations, namely $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline functions as a direct ablation of the proposed temporal factorization and closely resembles the formulation introduced in (Kollovieh et al., 2024); implementation details are provided in Appendix B.

Evaluation To assess extrapolation beyond the training distribution, each trajectory was partitioned into three temporally ordered segments: an observation window supplying the historical model input, a prediction window used during training as the supervised target, and an extrapolation window that remained entirely unseen during training. Forecast quality was evaluated with the continuous rank probability score (CRPS) (Winkler et al., 1996), which measures probabilistic calibration and distributional accuracy, and the normalized root mean square error (NRMSE) (Hindman & Koehler, 2006), which quantifies point-prediction accuracy after scale normalization.

Table 1: Performance comparison between the autoregressive (FLOWTIME) and non-autoregressive (Non-AR) factorizations on five stochastic dynamical systems. Test-set performance is reported using CRPS and NRMSE in both prediction and extrapolation regimes, with entries giving the mean and standard deviation over five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results The results in Table 1 indicate that the autoregressive factorization used by FLOWTIME substantially improves forecasting across the evaluated stochastic dynamical systems. In the prediction regime, the AR model attains lower NRMSE than the non-AR baseline for all five systems, with especially pronounced gains on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to error reductions of 93% and 95%, respectively. The same factorization also produces lower CRPS in four of the five systems, with Van der Pol being the sole case in which the non-AR baseline yields better uncertainty quantification. More consequentially, these advantages persist under extrapolation: the autoregressive model maintains superior NRMSE on every system, most strikingly for Brusselator, where extrapolation NRMSE is reduced by 98% (0.023 vs 1.046). Collectively, these findings support the interpretation that explicit temporal factorization enables FLOWTIME to capture the governing stochastic dynamics more faithfully, thereby improving both accuracy and uncertainty characterization in demanding extrapolative settings.

4.2 REAL-WORLD DATASETS

Data We further evaluated FLOWTIME on multiple univariate and multivariate timeseries forecasting tasks from GluonTS (Alexandrov et al., 2020), spanning both hourly and daily observation

frequencies. The benchmark suite comprises Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019), with supplementary dataset details reported in Appendix A.2.

Implementation details For the real-world datasets, ζ_ϕ was specified as a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ was implemented as a residual neural network containing five residual blocks built from 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020); this design is similar to the architecture in (Rasul et al., 2021), although it uses fewer residual blocks. Fourier positional embeddings encoded the flow step s into a 32-dimensional vector using a fixed frequency set, and the same embedding strategy was applied to time-dependent covariates c . Models were trained with Adam using a learning rate of 0.001 and a batch size of 128, with convergence times ranging from 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU, depending on the dataset.

Baselines We compared FLOWTIME with a heterogeneous set of established forecasting baselines, including classical statistical approaches—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008)—as well as neural forecasting models such as DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). The comparison also includes diffusion-based baselines, namely TSDiff (Kolloviah et al., 2023), SSSD (Alcaraz & Strodtzoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow-matching baseline TSFlow (Kolloviah et al., 2024).

Table 2: Test-set CRPS comparison (lower is better) for multiple forecasting models on five real-world datasets. FLOWTIME entries report the mean and standard error over five independent re-training and evaluation runs; baseline results are taken from (Kolloviah et al., 2024). Best scores are shown in **bold**, and second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 compares FLOWTIME with the selected baselines on five real-world timeseries datasets. The proposed model attains state-of-the-art CRPS on three datasets—Electricity, Exchange, and Solar. On Electricity, FLOWTIME achieves a CRPS of 0.042, improving on the second-best method, TSFlow (0.045), by 6.7%. On Exchange, it matches TSFlow at 0.009 while substantially outperforming the remaining statistical and neural baselines. The largest margin appears on Solar, where FLOWTIME obtains a CRPS of 0.284, corresponding to a 17.2% error reduction relative to TSFlow (0.343), the next-best model. Although FLOWTIME ranks second on Traffic, with a CRPS of 0.089 compared with TSFlow’s 0.083, it remains ahead of the other traditional and deep-learning alternatives; on Wikipedia, its CRPS of 0.243 is competitive but trails WaveNet (0.207) and SSSD (0.209). Overall, these results indicate that the autoregressive flow-matching formulation yields consistently strong, and frequently leading, performance across diverse real-world forecasting tasks.

5 DISCUSSION

We have introduced FLOWTIME, a probabilistic forecasting model that integrates autoregressive temporal factorization with flow matching in order to produce accurate and well-calibrated forecasts for multivariate timeseries. By reducing the forecast distribution to a sequence of conditional laws, the model combines an explicit representation of temporal dependence with the expressivity of flow-based generative modeling. The empirical results on both simulated dynamical systems and real-

world datasets show that FLOWTIME is competitive with, and in several settings superior to, state-of-the-art baselines on challenging forecasting benchmarks. Relative to standard flow-matching approaches that model complete future windows simultaneously, the autoregressive decomposition yields notable gains in uncertainty calibration and distributional coverage. In addition, FLOWTIME demonstrates robust generalization in out-of-distribution extrapolation regimes, a property that is especially important for practical deployment when test-time conditions need not coincide with the training distribution.

Several limitations nevertheless remain. First, because sampling proceeds sequentially, autoregressive generation incurs greater inference-time overhead than fully parallelizable non-autoregressive models, which may constrain use in stringent real-time forecasting systems. Second, the present implementation performs flow matching directly in data space, an approach that may be suboptimal for high-dimensional or partially observed processes. Future work could therefore investigate latent-space variants, extensions to irregularly sampled observations, and broader applications in scientific modeling and decision-making under uncertainty. Taken together, the results position FLOWTIME as a meaningful step toward more expressive probabilistic timeseries forecasting, illustrating how flow-based generative models can improve both predictive accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

-
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.
- Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
- Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, 2018.
- R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.
- Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
- Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.
- Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.
- Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.
- Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media foundation models. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint arXiv:2002.06103*, 2020.
- Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*, pp. 8857–8868. PMLR, 2021.
- David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus. High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances in Neural Information Processing Systems*, 32, 2019.

-
- David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3): 1181–1191, 2020.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pp. 2256–2265. PMLR, 2015.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow matching for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Generative Modeling*, 2024.
- Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. *Advances in Neural Information Processing Systems*, 34:24804–24816, 2021.
- Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrod Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
- Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.
- B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
- Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.
- Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time series foundation models: Generalizing time series representation with large language model. In *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

Training and testing data were generated for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating the corresponding stochastic differential equations (SDEs). Each system is described by the generic form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t); \Theta) dt + \sigma d\mathbf{W}(t),$$

where f denotes the deterministic drift, Θ collects the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. For each system, 2400 initial conditions were sampled uniformly from the specified range in Table 3, after which one trajectory per initial condition was produced by solving the SDE with the fixed-step Euler-Heun method. Each trajectory was observed at 200 equally spaced time points; the first 75 served as observed context, the next 75 as the prediction segment, and the final 50 as the extrapolation segment. Of the 2400 generated trajectories, 2000 were allocated to training and 400 to testing.

Table 3: Summary of the stochastic dynamical systems used for simulation. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial-condition distribution; each system was integrated over $[t_0, t_1]$ using 200 Euler-Heun steps.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\beta = 0.9$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

Model performance was evaluated on five datasets from GluonTS (Alexandrov et al., 2020), using the versions preprocessed according to (Salinas et al., 2019). Their salient properties are summarized in Table 4.

Table 4: Summary properties of the real-world datasets used in the experiments, including dimension n , domain Ω , sampling frequency, total number of training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We consider the same forecasting problem as in Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the objective is to model $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to the autoregressive formulation, however, this baseline represents the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ as a single joint random object without temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves cross-temporal correlations over the full horizon, but it correspondingly requires estimation of a substantially higher-dimensional distribution.

Training We construct a trajectory-level probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from the prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is block diagonal. This prior may be interpreted as independent Brownian motion processes across dimensions, with Brownian motion viewed as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

The conditional probability path is specified through a linear interpolation bridge augmented with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$, $\mathbf{Y}_f^0 \sim p^0$, and $\mathbf{Y}_f^1 \sim p^1$. The induced conditional velocity field is

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

In the limit $\sigma^2 \rightarrow 0$, this expression reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We then learn a neural velocity field ν_θ acting on the entire future trajectory, while retaining a context encoder analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both historical and future covariates. The training objective is therefore

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network, such as a Transformer or RNN, that processes the complete trajectory. Sampling consequently requires solving the trajectory-level ODE

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with numerical solvers, producing joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without invoking an autoregressive decomposition.

Implementation Details This baseline was used in the dynamical-systems forecasting experiments. In this configuration, ν_θ was specified as a 4-layer bi-directional LSTM with 128 hidden units, while all remaining training parameters matched those in Section 4.1; convergence required approximately ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs in several fundamental respects from the autoregressive model introduced in FLOWTIME. First, whereas the autoregressive method factorizes the joint law through a Markovian decomposition over time, the non-autoregressive baseline represents the entire future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, preserving cross-temporal dependencies while requiring the estimation of a more complex distribution over $\mathbb{R}^{n \times f}$. Second, the computational profiles diverge: the non-autoregressive model processes full trajectories through sequential velocity-field networks and permits parallel generation of all future time points, but this parallelism is obtained at the cost of greater memory demand. Finally, the two approaches encode uncertainty propagation differently. The autoregressive variant explicitly models the accumulation and transformation of predictive errors through a chain of conditional distributions, whereas the non-autoregressive baseline captures joint uncertainty through trajectory-level sampling without an explicit mechanism for error-accumulation dynamics. These distinctions yield complementary trade-offs: the autoregressive model provides interpretable uncertainty quantification and efficient window-based computation under finite-order Markov assumptions, while the non-autoregressive baseline preserves full temporal correlations at higher computational cost and with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section specifies the metrics used to evaluate forecast quality. Given an n -dimensional forecast horizon $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ together with either its predictive distribution or samples generated by the probabilistic model, we report two complementary criteria: the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely used to evaluate probabilistic forecasts because it rewards both calibration and sharpness. For a univariate random variable X with cumulative distribution function (CDF) F_X and observed realization x , it is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ denotes the indicator function. Intuitively, CRPS measures the discrepancy between the full predictive distribution and the realized observation x , so smaller values correspond to better calibrated and more accurate distributional forecasts.

For timeseries forecasting, CRPS is computed at each horizon step and then averaged over the forecast window:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ denotes the predicted CDF, or the empirical CDF induced by samples, at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth value.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard criterion for assessing point-forecast accuracy. Given predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is defined as

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To render RMSE scale-invariant, and thus comparable across datasets or series with different magnitudes, we report the Normalized RMSE (NRMSE). A common normalization divides by the standard

deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Depending on the application, normalization by the empirical range ($\max(y) - \min(y)$) is also possible; under either convention, lower NRMSE denotes more accurate predictions relative to the intrinsic variability of the series.